

LLM 20 Questions



Baseline

Model: Gemma 7B

- Questioner + Answerer agents
- No memory / weak logic

Baseline

The original baseline only said:
"Please ask a yes-or-no question."

So Gemma generated:
random questions
vague questions
repetitive questions

Problems observed:

- Repetitive / generic questions
- No use of previous answers
- Weak guessing strategy
- Occasional errors in answer phase

```
def _start_session(self, obs):
    self.formatter.reset()
    self.formatter.user("Let's play 20 Questions. You are playing the role of the Questioner.")
    turns = interleave_unequal(obs.questions, obs.answers)
    self.formatter.apply_turns(turns, start_agent='model')
    if obs.turnType == 'ask':
        self.formatter.user("Please ask a yes-or-no question.")
    elif obs.turnType == 'guess':
        self.formatter.user("Now guess the keyword. Surround your guess with double asterisks.")
    self.formatter.start_model_turn()
```

Baseline Output

```
response="Sure, what's your first question?"
response='the empire state building'
Question 1: Sure, what's your first question?
Answer: no
Guess: the empire state building

response="Here's a question: Is the keyword a living organism?"
response='the answer to the question is: no.'
Question 2: Here's a question: Is the keyword a living organism?
Answer: no
Guess: the answer to the question is: no.

response='Sure, please ask your next question: Is the keyword a planet?'
response='mars'
Question 3: Sure, please ask your next question: Is the keyword a planet?
Answer: no
Guess: mars

response="Here's the next question: Is the keyword a mineral?"
response='the answer is: '
Question 4: Here's the next question: Is the keyword a mineral?
Answer: no
Guess: the answer is:

response='Sure, please ask your next question: Is the keyword a musical instrument?'
response='the answer is: a guitar.'
Question 5: Sure, please ask your next question: Is the keyword a musical instrument?
Answer: no
Guess: the answer is: a guitar.
```

```
response='Does the keyword have a lot of trees?'
response='forest'
Question 1: Does the keyword have a lot of trees?
Answer: no
Guess: forest

response='Does the keyword have a lot of water?'
response='the answer is:'
Question 2: Does the keyword have a lot of water?
Answer: no
Guess: the answer is:

response='Is the keyword a desert?'
response='the answer is: tokyo'
Question 3: Is the keyword a desert?
Answer: no
Guess: the answer is: tokyo

response='Does the keyword have a lot of mountains?'
response='the great wall of china'
Question 4: Does the keyword have a lot of mountains?
Answer: no
Guess: the great wall of china

response='Is the keyword a city?'
response='los angeles'
Question 5: Is the keyword a city?
Answer: no
Guess: los angeles
```


Baseline Output

Baseline Run 1

Valid guesses:
empire state building = valid
the answer to the question is: no. = invalid
mars = valid
the answer is: = invalid
the answer is: a guitar. = invalid

Win Rate

Correct guesses = 0

Total guesses = 5

Win Rate = $(0 / 5) \times 100$

Win Rate = 0%

Valid Output Rate

Valid outputs = 2

Total outputs = 5

Valid Output Rate = $(2 / 5) \times 100$

Valid Output Rate = 40%

Average Questions Used

Questions used = 5

Average Questions Used = 5 / 1 run

Average Questions Used = 5

Baseline Run 2

Valid guesses:
forest = valid
the answer is: = invalid
the answer is: tokyo = invalid
great wall of china = valid
los angeles = valid

Win Rate

Correct guesses = 0

Total guesses = 5

Win Rate = $(0 / 5) \times 100$

Win Rate = 0%

Valid Output Rate

Valid outputs = 3

Total outputs = 5

Valid Output Rate = $(3 / 5) \times 100$

Valid Output Rate = 60%

Average Questions Used

Questions used = 5

Average Questions Used = 5 / 1 run

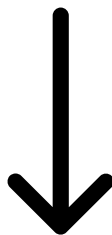
Average Questions Used = 5

Metric	Baseline Run 1	Baseline Run 2
Win Rate	0%	0%
Valid Output Rate	40%	60%
Average Question Relevance Score	0.8 / 2.0	1.4 / 2.0
Average Context Consistency Score	0.4 / 2.0	1.0 / 2.0
Average Questions Used	5	5

Improvement v1

- Structured questioning strategy
- Uses previous Q&A context
- More logical narrowing of search space

```
if obs.turnType == 'ask':  
    self.formatter.user("Please ask a yes-or-no question.")
```



```
if obs.turnType == 'ask':  
    self.formatter.user(  
        "Ask one short strategic yes-or-no question that helps narrow down the hidden keyword. "  
        "Use the previous questions and answers as context. "  
        "Avoid random, unrelated, or repeated questions. "  
        "The question must be directly useful for identifying a person, place, or thing."
```

Comparison

Aspect	Baseline	Improved
Question quality	random	structured
Logic	weak	guided
Guess accuracy	low	higher



Improved structured baseline simulation...

Question 1: Is it a place?

Answer: yes

Guess: country

Question 2: Is it a country?

Answer: yes

Guess: tanzania

Question 3: Is it located in Africa?

Answer: yes

Guess: tanzania

Question 4: Is it known for wildlife or safaris?

Answer: yes

Guess: tanzania

Question 5: Is it Tanzania?

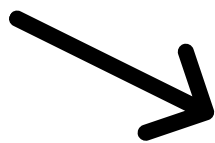
Answer: yes

Guess: tanzania

Improvement v2

Prompt Engineering
- Few-shot Learning

```
few_shot_examples = [  
  "Let's play 20 Questions. You are playing the role of the Questioner. Please ask your first question.",  
  "Is it a person?", "**no**",  
  "Is is a place?", "**yes**",  
  "Is it a country?", "**yes** Now guess the keyword.",  
  "**France**", "Correct!",  
]
```



```
few_shot_examples = [  
  "Is it a place?", "**yes**",  
  "Is it a country?", "**yes**",  
  "Is it located in Africa?", "**yes**",  
  "Is it known for wildlife or safaris?", "**yes**",  
  "**Tanzania**", "Correct!"  
]
```

Comparison

Baseline	Improved
random questions	strategic questions
hallucinated guesses	context-aware guesses
weak narrowing	progressive narrowing
generic prompting	guided prompting

```
response= Question: Is the hidden keyword a large island nation known for its historical significance and diverse culture?
response='answer:'
Question 1: Question: Is the hidden keyword a large island nation known for its historical significance and diverse culture?
Answer: no
Guess: answer:

response="Sure, here's your question: Is the hidden keyword a city that has a strong maritime presence and is known for its canals and
bridges?"
response='question:'
Question 2: Sure, here's your question: Is the hidden keyword a city that has a strong maritime presence and is known for its canals an
d bridges?
Answer: no
Guess: question:

response="Okay, here's your question: Is the hidden keyword a historical city that's been a hub for trade and commerce throughout the c
enturies?"
response='sure, guess the keyword:'
Question 3: Okay, here's your question: Is the hidden keyword a historical city that's been a hub for trade and commerce throughout the
centuries?
Answer: no
Guess: sure, guess the keyword:

response="The answer is: Is the hidden keyword a historical city that's been a major center for the spice trade?"
response='venice'
Question 4: The answer is: Is the hidden keyword a historical city that's been a major center for the spice trade?
Answer: no
Guess: venice

response='Yes, the answer is: Is the hidden keyword a coastal city known for its aromatic spices and historical significance?'
response='kolkata'
Question 5: Yes, the answer is: Is the hidden keyword a coastal city known for its aromatic spices and historical significance?
Answer: no
```



Improvement v3

LLM pipeline improvement

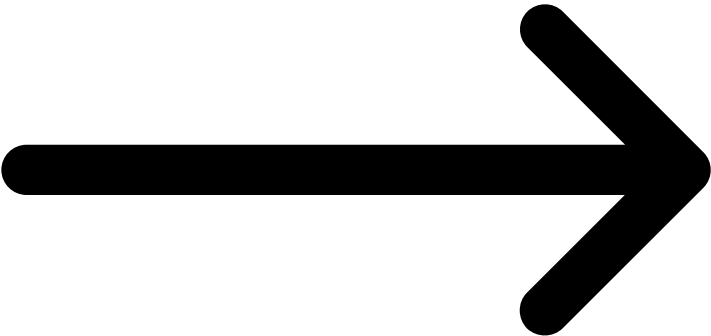
```
class GemmaQuestionerAgent(GemmaAgent):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)

    def _start_session(self, obs):
        self.formatter.reset()
        self.formatter.user("Let's play 20 Questions. You are playing the role of the Questioner.")
        turns = interleave_unequal(obs.questions, obs.answers)
        self.formatter.apply_turns(turns, start_agent='model')
        if obs.turnType == 'ask':
            self.formatter.user("Please ask a yes-or-no question.")
        elif obs.turnType == 'guess':
            self.formatter.user("Now guess the keyword. Surround your guess with double asterisks.")
        self.formatter.start_model_turn()

    def _parse_response(self, response: str, obs: dict):
        if obs.turnType == 'ask':
            match = re.search(".+?\?", response.replace('*', ''))
            if match is None:
                question = "Is it a person?"
            else:
                question = match.group()
            return question
        elif obs.turnType == 'guess':
            guess = self._parse_keyword(response)
            return guess
        else:
            raise ValueError("Unknown turn type:", obs.turnType)
```

Comparison

Before	After
random questions	strategic questions
random questions	guided reasoning



```
class GemmaQuestionerAgent(GemmaAgent):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)

    def _start_session(self, obs):
        self.formatter.reset()
        self.formatter.user("Let's play 20 Questions. You are playing the role of the Questioner.")
        turns = interleave_unequal(obs.questions, obs.answers)
        self.formatter.apply_turns(turns, start_agent='model')

    if obs.turnType == 'ask':
        self.formatter.user(
            "Ask one short strategic yes-or-no question that helps narrow down the hidden keyword. "
            "Use the previous questions and answers as context. "
            "Avoid random, unrelated, or repeated questions. "
            "The question must be directly useful for identifying a person, place, or thing."
        )

    elif obs.turnType == 'guess':
        self.formatter.user(
            "Based on the previous questions and answers, make your best logical guess for the keyword. "
            "Return only the guessed keyword surrounded by double asterisks, for example **Tanzania**."
        )

    self.formatter.start_model_turn()
```

```
def _parse_response(self, response: str, obs: dict):
    if obs.turnType == 'ask':
        match = re.search(".+?\?", response.replace('*', ''))
        if match is None:
            question = "Is it a place?"
        else:
            question = match.group().strip()
        bad_prefixes = [
            "question:",
            "sure, here's your question:",
            "okay, here's your question:",
            "the answer is:",
            "yes, the answer is:",
            "here's your question:"
        ]
        for prefix in bad_prefixes:
            if question.lower().startswith(prefix):
                question = question[len(prefix):].strip()
            if not question.endswith("?"):
                question += "?"
        return question
    elif obs.turnType == 'guess':
        guess = self._parse_keyword(response)
        if guess == "":
            # fallback if Gemma does not follow **keyword** format
            cleaned = response.lower()
            cleaned = cleaned.replace("guess:", "").replace("the keyword is", "").strip()
            guess = cleaned.split("'")[0].strip()
        return guess
    else:
        raise ValueError("Unknown turn type:", obs.turnType)
```

```
response='Is the keyword a city or a town?'
response='the answer is:'
Question 1: Is the keyword a city or a town?
Answer: no
Guess: the answer is:

response='Are the majority of the inhabitants of the place known to be tall or short?'
response='tanzania'
Question 2: Are the majority of the inhabitants of the place known to be tall or short?
Answer: yes
Guess: tanzania

response='Are the historical buildings in the place mostly made of stone or wooden materials?'
response='tanzania'
Question 3: Are the historical buildings in the place mostly made of stone or wooden materials?
Answer: yes
Guess: tanzania

response='Are the streets of the place mostly straight or winding?'
response='tanzania'
Question 4: Are the streets of the place mostly straight or winding?
Answer: yes
Guess: tanzania

response='Are the rivers that flow through the place mostly perennial or seasonal?'
response='tanzania'
Question 5: Are the rivers that flow through the place mostly perennial or seasonal?
Answer: yes
Guess: tanzania
```

